Feasibility Decides the Trade-off: Injecting a Constructive Prior into Multi-Objective Spacecraft Payload Packing

Authors: [Author names, affiliations and contact details to be supplied by the authors]

Abstract

Stowing cargo in a spacecraft logistics module couples a three-dimensional packing problem with two conflicting goals: keeping the centre of mass (CoM) near its target and keeping items accessible to the crew. We study this problem as bi-objective search with NSGA-II over packing sequences and orientations, decoded by an Extreme Point heuristic. The test set has twelve synthetic instances of 50 to 500 items and six instances derived from the Bischoff–Ratcliff container-loading benchmark. All methods are compared under a matched evaluation budget with 30 seeds per synthetic instance, seed-paired tests and Holm correction, in about 2,100 runs.

On the synthetic instances, plain NSGA-II rarely returns a valid packing. It succeeds in at most 60% of seeds, and in at most 4 of 30 seeds on five of eight solvable instances. A decreasing-volume constructive order restores feasibility, and we compare two ways of injecting it:

- **A Baldwinian repair** that re-decodes failed chromosomes in volume order. It is feasible in 87–100% of seeds and beats plain NSGA-II in hypervolume on all eight instances.
- **Seeding** a single best-fit-decreasing solution into the initial population. This is significantly better than the repair on five of eight instances, including every instance with 250 or 500 items, and improves both objectives there. It uses half the decoder calls. The repair is better on the two smallest consumable-heavy instances.

On seeds where both methods are feasible, the repair does not improve hypervolume over NSGA-II. It shifts the front toward lower CoM deviation and lower accessibility.

On the Bischoff–Ratcliff instances, the prior's value depends on cargo heterogeneity: - With 3 box types, plain NSGA-II is usually feasible and the repair lowers front quality. - With 20 box types, NSGA-II never finds a valid packing and the repair is necessary.

An ablation traces the gain to the volume order, and a contact constraint reduces feasibility for every method. We conclude that evaluations of packing optimisers should report feasibility separately from front quality, include strong constructive baselines, and treat how a prior is injected, and whether one is needed at all, as a design choice to be tested.

Keywords: spacecraft logistics; three-dimensional packing; container loading; multi-objective evolutionary algorithms; constraint handling; repair; population seeding; NSGA-II; hypervolume

1. Introduction

Cargo for a crewed spacecraft must be packed so that the vehicle's centre of mass stays within tolerance. It must also remain retrievable by a crew who reach in through an access face and may have to remove items to reach the one they need. The two goals conflict. Compact packings that keep the CoM on target tend to bury items, and accessible packings tend to load mass unevenly. Multi-objective evolutionary algorithms (MOEAs) are well suited to this, because they return a set of compromises rather than a single packing.

A decoder-based MOEA searches over packing sequences and orientations, and a placement heuristic converts each chromosome into a layout. This design is standard in container loading (Gonçalves and Resende, 2013), but it has a weakness that is easy to overlook. The decoder may fail to place every item, so a chromosome can describe an invalid packing. Constrained-domination selection (Deb, 2000; Deb et al., 2002) steers the search toward feasibility but does not guarantee it. An

evaluation that compares fronts without separating feasibility either compares invalid layouts or silently rewards whichever method finds any valid one.

Constructive heuristics such as best-fit-decreasing by volume handle feasibility well, but return a single layout. An obvious remedy is to inject the constructive order into the MOEA. It can enter at decoding time, by repairing failed chromosomes (Salcedo-Sanz, 2009), or at initialisation, by seeding the population with the heuristic's solution (Friedrich and Wagner, 2015). This paper asks whether these remedies work for CoM–accessibility packing, why they work, when they are needed, and what they cost the trade-off. Our contributions are:

1. **An evaluation protocol that separates feasibility from front quality.** It uses matched evaluation budgets, pooled feasible reference fronts, feasible-run rates, seed-paired tests with family-wise correction, and a comparison restricted to seeds on which both methods are feasible (Sections 4 and 6.1).
2. **A comparison of two ways to inject the same constructive prior,** a Baldwinian volume-ordered repair and one seeded individual. They are compared against plain NSGA-II, random search and three deterministic heuristics on eight solvable synthetic instances with up to 500 items and 30 seeds each. Seeding wins at scale and the repair wins on the smallest instances (Section 6.1).
3. **Evidence on where the gain comes from, what it costs, and when it is needed:**
   - a feasibility-conditioned analysis showing that the repair shifts the front toward CoM compactness at the expense of accessibility (Section 6.3);
   - an ablation that attributes the gain to the volume order and shows orientation search is essential (Section 6.5);
   - a contact-constrained decoder that shows how the conclusions change when items may not float (Section 6.6);
   - an external benchmark derived from the Bischoff–Ratcliff instances, which shows that the value of the prior grows with cargo heterogeneity and that it can hurt on near-homogeneous cargo (Section 6.7);
   - a sensitivity analysis showing that the main comparison is robust to the accessibility constants (Section 6.4).
4. **A corrected and tested batched GPU scorer.** The original scorer treated padding rows as items, which biased the accessibility objective of every infeasible layout. The corrected scorer is 4–6× faster than the CPU reference for scoring (Section 6.8).

We do not validate the accessibility surrogate against crew data, model launch loads, or claim a whole-run GPU speedup. Section 8 states these limits.

2. Related Work

Container loading. Bischoff and Ratcliff (1995) set out the practical issues that distinguish container loading from abstract packing. They also introduced the benchmark instances we adapt in Section 6.7. Bortfeldt and Wäscher (2013) review the constraints that arise in practice, including weight distribution, stability, orientation, loading priorities and positioning. In their terms, our CoM objective is a weight-distribution criterion and our accessibility objective is a positioning criterion. Davies and Bischoff (1999) give an early treatment of weight distribution in container loading. Ramos, Oliveira, Gonçalves and Lopes (2016) enforce static mechanical equilibrium, a stricter stability model than the contact constraint we test. Ceschia and Schaerf (2013) address multi-drop loading, where items must be reachable in delivery order, an accessibility requirement related to ours. Zhao, Bennell, Bektaş and Dowsland (2016) compare container loading algorithms on common benchmarks.

Three-dimensional packing heuristics. Martello, Pisinger and Vigo (2000) established exact and heuristic methods for three-dimensional bin packing. Crainic, Perboli and Tadei (2008) introduced Extreme Points (EP) as placement candidates, which our decoder uses. Parreño, Alvarez-Valdes, Tamarit and Oliveira (2008) use maximal spaces instead. Gonçalves and Resende (2013) encode a

packing as a sequence and orientation vector decoded by a placement heuristic and evolved with a biased random-key genetic algorithm. Our representation follows this decoder-based design.

Space cargo. Cargo accommodation for the International Space Station has been supported by an operational tool, CAST, which combines global optimisation with heuristics and balancing rules (Fasano, Lavopa, Negri and Vola, 2015). Fasano (2014) treats non-standard packing problems from this domain. Ho (2023) reviews space logistics optimisation at the campaign and network level. We found no study that pairs a CoM objective with an accessibility objective under an EP decoder and compares methods under a matched budget. This reflects our search, not a guarantee that none exists.

Constraint handling, repair and seeding in evolutionary algorithms. NSGA-II (Deb et al., 2002) handles constraints by constrained domination, following Deb's (2000) feasibility rules. Repair maps an infeasible solution to a feasible or less infeasible one, and Salcedo-Sanz (2009) surveys repair methods as constraint-handling techniques. When the repaired phenotype is evaluated but not written back to the genotype, the repair is Baldwinian rather than Lamarckian (Whitley, Gordon and Mathias, 1994). Our repair is Baldwinian. Seeding inserts known good solutions into the initial population. Friedrich and Wagner (2015) show that it can speed up MOEAs considerably, and also that it can bias the final population. We use the pymoo implementation of NSGA-II (Blank and Deb, 2020).

Evaluation methodology. Hypervolume (Zitzler and Thiele, 1999) and the wider methodology of quality indicators (Zitzler, Thiele, Laumanns, Fonseca and da Fonseca, 2003) underpin our front comparisons. For comparing stochastic algorithms over several problems we follow Derrac, García, Molina and Herrera (2011): per-problem nonparametric tests with a family-wise correction across problems. We use the procedure of Holm (1979) for the correction and Cliff's (1993) δ as the effect size.

GPU evaluation. Özcan, Dokeroğlu, Coşar and Yazıcı (2016) run a grouping genetic algorithm for one-dimensional bin packing on a GPU. Our batched scorer addresses three-dimensional objectives and is reported only as a scoring-stage measurement.

3. Problem and Methods

3.1 Representation and decoder

A solution is a permutation σ of the N items together with an orientation vector $o \in \{1,\ldots,6\}^N$. Each orientation selects one of the six axis permutations of an item's width, depth and height. Where an item may use only some orientations (Section 6.7), the gene indexes its permitted orientations cyclically. With all six permitted, the gene maps to itself.

The Extreme Point decoder places items in the order σ. Each item is tried, in its chosen orientation, at the current extreme points sorted by z, then y, then x. It is placed at the first point where it lies inside the container and overlaps no placed item, and the extreme points are then updated. An item with no valid point is left unplaced.

The access face is the plane z = 0, and z is depth from that face. The decoder therefore fills the container from the access face inward. By default it does not require an item to touch the items nearer the access face. Section 6.6 studies a contact-constrained variant.

3.2 Constraints and objectives

A layout is feasible when every item is placed. The constraints are g_1 = total unplaced volume ≤ 0 and g_2 = number of unplaced items ≤ 0.

- **f_1, CoM deviation (minimise).** The Euclidean distance between the system centre of mass and a target point t, where the system includes a dry mass of 5000 kg located at t. On the synthetic

instances, t = (0.5, 0.5, 1.0) m. Because this target is absolute and container sides range from 1.37 to 3.17 m, Section 6.4 also tests a target at the container centre. On the Bischoff–Ratcliff instances, t is the container centre.

- **f_2, accessibility (maximise; implemented as minimising −CASC).** For each packed item i, CASC_i = exp(−c_i), with c_i = c_reach + c_occ + c_ext.
  - c_reach = (z_i / L_arm)² with L_arm = 0.75 m.
  - c_occ = λ(1 − v_i) with λ = 1, where v_i is the visible fraction of item i's footprint on a 10 × 10 grid, counting only items nearer the access face.
  - c_ext = Σ_j ρ_j (C_grasp + κ m_j) over the transitive blockers j of i, with C_grasp = 2, κ = 0.05 kg⁻¹, and ρ_j = 0.4 for consumable or waste items and 1 otherwise.

  The objective is the mean CASC over packed items.

A layout with no packed item receives $f_1 = 9999$ and $f_2 = 0$.

3.3 Optimiser

We use NSGA-II with constrained domination on (g_1, g_2). Population and generations are 100 and 100 for N ≤ 111, and 50 and 50 for N ≥ 250. The operators are:

- **Crossover:** OX1 order crossover on σ and uniform crossover on o (mask 0.5), applied with probability 0.9.
- **Mutation:** swap or inversion on σ, and reset on each orientation gene, each with probability 1/N.
- **Duplicates:** eliminated.

3.4 Injecting the volume-order prior

Each of the variants below changes exactly one thing relative to plain NSGA-II.

Volume repair (Baldwinian). If decoding leaves an item unplaced, the chromosome is re-decoded with σ stably sorted by decreasing item volume and the same orientations. The decode with less unplaced volume is kept, and the genotype is not modified.

```
evaluate(σ, o):
    layout ← EP-decode(σ, o)
    if layout leaves an item unplaced:
        layout' ← EP-decode(sort σ by decreasing volume, o)
        if unplaced-volume(layout') < unplaced-volume(layout): layout ← layout'
    return f₁, f₂, g₁, g₂ of layout          # σ and o are not changed
```

Large-item repair. As the volume repair, except that only items with volume at or above the instance median move to the front, in decreasing volume order. The smaller items keep their order from σ. The intent is to secure feasibility with the large items while the search keeps control of the small items, and so of accessibility.

Seeded NSGA-II. Plain NSGA-II whose initial population contains one BFD-V chromosome: the decreasing-volume sequence with the orientations BFD-V chose (Section 3.5). The other individuals are random. A test verifies that the seed decodes to exactly the BFD-V layout. Where BFD-V is infeasible, the seed is still the BFD-V order, which fails to place every item.

The two repairs cost one or two decoder calls per evaluation. In practice the first decode almost always fails, so they used 19,900–20,000 decoder calls per 10,000 evaluations. Seeding uses exactly as many decoder calls as plain NSGA-II.

3.5 Baselines

- **Plain NSGA-II.**

- **Random search,** with the same number of evaluations as NSGA-II on the same instance and seed.
- **Three deterministic constructions** with the same EP decoder. Unlike the MOEA decode, they try every permitted rotation at each extreme point and take the first fit. First-Fit (FF) uses dataset order, BFD-V uses decreasing volume, and BFD-P uses decreasing priority.

4. Experimental Design

Synthetic instances. Twelve instances (HyperDock-Bench V1) with N ∈ {50, 100, 250, 500} in three scenarios:

- SC1 is consumable-heavy, at 80% volume utilisation.
- SC2 is safety-item-heavy (40%), at 80% utilisation.
- SC3 has the SC1 mix at 96% utilisation.

Item sizes follow cargo-transfer-bag classes plus an irregular class. Masses are lognormal by category, and each container is a cube sized to the target utilisation. Every instance is seeded from its identifier, and the committed generator reproduces all twelve files exactly. Distributions and per-instance statistics are in `docs/DATASETS.md`.

External instances. Section 6.7 describes six instances derived from the Bischoff–Ratcliff benchmark.

Runs. On the eight SC1 and SC2 instances, every method is run on seeds 1–30: - NSGA-II, the volume repair and random search (720 runs); - seeded NSGA-II and the large-item repair (480 runs).

On SC3 every method is infeasible, so its 250- and 500-item instances were run with seeds 1–10 only. Together with SC3, the three-method study totals 960 runs. The remaining runs are: - the 36 deterministic constructions on the synthetic instances; - 120 ablation runs (Section 6.5); - 204 contact-decoder runs (Section 6.6); - 306 runs on the Bischoff–Ratcliff instances (Section 6.7).

That makes about 2,100 runs in total. Every evaluated chromosome is archived, so comparisons use all evaluations rather than only the final population. All objective values come from the CPU reference scorer.

Platform. A laptop with 28 logical CPU cores and an NVIDIA RTX 4050 Laptop GPU, running Python 3.12, pymoo 0.6.2, numpy 2.5.4, scipy 1.18.1 and numba 0.68.1.

Metrics.

- **Feasible-run rate:** the fraction of seeds with at least one feasible evaluation.
- **Hypervolume (HV):** per instance, the feasible points of all methods and seeds in a study are pooled, the objectives are min-max normalised on that pool, and HV is taken with reference point (1.1, 1.1). A run with no feasible point has HV 0. HV values are comparable within an instance and a study, not across studies with different method sets.
- **IGD:** distance to the non-dominated pooled set, defined for feasible runs only.

Statistics. Each pair of methods is compared per instance with a two-sided Wilcoxon signed-rank test paired by seed. Effect size is Cliff's δ, and intervals are bootstrap 95% intervals of the mean (10,000 resamples). Holm correction is applied across instances within each comparison. Comparisons where both methods have HV 0 in every run are excluded from the family, because no test is possible there; this covers all of SC3. With n seeds, the smallest attainable p-value is $2/2^n$: 0.002 for 10 seeds and 1.9×10^{-9} for 30.

5. Implementation Correctness

Batched-scorer defect. A batched PyTorch scorer packs a population on the CPU and evaluates the accessibility objective on the GPU. It passed unplaced items as zero rows and scored them as real

items at the origin, with errors up to about 0.1 in mean CASC. Fully packed layouts were unaffected, so an equivalence check on those passed, but most layouts visited during search are partially packed. The corrected scorer masks the padding rows and matches the CPU reference to within 8×10^{-8} on partially packed layouts. Regression tests cover that case. An earlier analysis of this system used the defective scorer and is withdrawn (Appendix A).

Cross-environment reproducibility. Pareto points from the laptop runs were re-decoded and re-scored in a separate Linux environment with different package versions, and the objective values were identical. The deterministic heuristics produced identical layouts in both environments. The test suite (50 tests) checks four things: - the generic implementation of every variant reproduces the published methods exactly; - the seeded chromosome decodes to the BFD-V layout; - every Bischoff–Ratcliff box respects its orientation flags; - the scorer correction holds.

6. Results

6.1 Feasibility and hypervolume on the synthetic instances

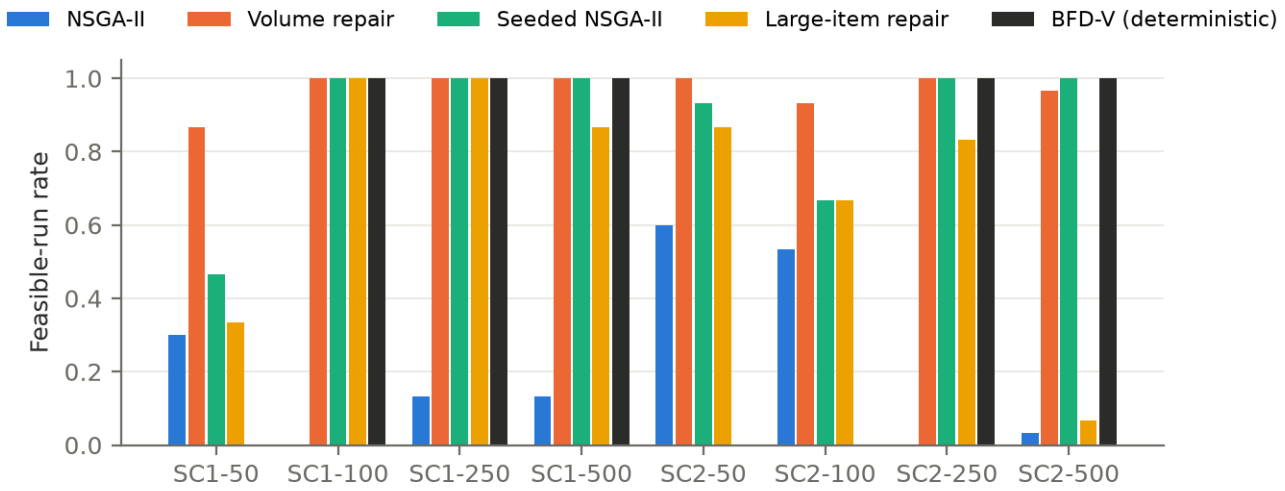


Figure 1: Feasible-run rate per instance and method (30 seeds). Random search, FF and BFD-P are infeasible everywhere and are omitted.

Table 1. Hypervolume, as the mean with its bootstrap 95% interval, and number of feasible seeds out of 30, per method. The reference front pools all methods. The best mean per row is in bold. The p-values are Holm-adjusted Wilcoxon tests across the eight instances, and the last column names the winner where the difference is significant.

Instance	NSGA-II	Volume repair	Seeded NSGA-II	Large-item repair	Repair vs NSGA-II	Seeded vs NSGA-II	Repair vs seeded
SC1-50	0.100 [0.043, 0.166] · 9	**0.371** [0.300, 0.437] · 26	0.211 [0.127, 0.305] · 14	0.112 [0.054, 0.177] · 10	4.0×10^{-4}	0.087	**0.046** (repair)
SC1-100	0 · 0	**0.608** [0.565, 0.659] · 30	0.291 [0.232, 0.354] · 30	0.498 [0.447, 0.547] · 30	1.5×10^{-8}	6.9×10^{-6}	**9.8×10^{-7}** (repair)
SC1-250	0.072 [0.014, 0.145] · 4	0.327 [0.308, 0.347] · 30	**0.679** [0.626, 0.735] · 30	0.296 [0.269, 0.320] · 30	2.5×10^{-4}	2.6×10^{-8}	**1.5×10^{-8}** (seeded)
SC1-500	0.095 [0.023, 0.191] · 4	0.237 [0.226, 0.250] · 30	**0.560** [0.466, 0.651] · 30	0.152 [0.124, 0.179] · 26	0.027	1.3×10^{-5}	**4.0×10^{-6}** (seeded)

Instance	NSGA-II	Volume repair	Seeded NSGA-II	Large-item repair	Repair vs NSGA-II	Seeded vs NSGA-II	Repair vs seeded
SC2-50	0.210 [0.138, 0.282] · 18	0.360 [0.317, 0.402] · 30	**0.474** [0.398, 0.546] · 28	0.303 [0.244, 0.360] · 26	0.012	4.8×10^{-5}	**0.013** (seeded)
SC2-100	0.270 [0.173, 0.369] · 16	**0.391** [0.328, 0.449] · 28	0.322 [0.230, 0.413] · 20	0.343 [0.246, 0.441] · 20	0.038	0.33	0.29
SC2-250	0 · 0	0.395 [0.360, 0.429] · 30	**0.682** [0.617, 0.750] · 30	0.291 [0.232, 0.349] · 25	1.5×10^{-8}	1.5×10^{-8}	**2.4×10^{-6}** (seeded)
SC2-500	0.014 [0.000, 0.042] · 1	0.142 [0.124, 0.158] · 29	**0.651** [0.560, 0.744] · 30	0.045 [0.000, 0.113] · 2	2.5×10^{-4}	2.6×10^{-8}	**1.5×10^{-8}** (seeded)

Feasibility.

- **Plain NSGA-II rarely finds a valid packing.** It succeeds in at most 60% of seeds on any instance. It finds none on SC1-100 and SC2-250, and at most 4 of 30 on SC1-250, SC1-500 and SC2-500. Random search never succeeds.
- **The volume repair** is feasible in 87–100% of seeds on every instance.
- **Seeding** is feasible in all 30 seeds wherever its seed is feasible. Where the seed is not, seeding still raises the feasible count above plain NSGA-II: 14 against 9 on SC1-50, 28 against 18 on SC2-50, and 20 against 16 on SC2-100.
- **The large-item repair** loses feasibility at 500 items, with 26 and 2 feasible seeds.

Front quality.

- **Both priors beat plain NSGA-II.** After Holm correction, the volume repair beats it on all eight instances, and seeding on six.
- **Between the two priors, seeding wins at scale.** It is significantly better on five instances: all four with 250 or 500 items, plus SC2-50. The effect sizes are large ($|\delta| \geq 0.39$, and ≥ 0.66 at $N \geq 250$).
- **The repair wins on the smallest consumable-heavy instances,** SC1-50 and SC1-100 ($\delta = 0.33$ and 0.81).
- **SC2-100 is a tie.** Neither prior is significantly better there.
- **The large-item repair is never best.** It is significantly worse than the volume repair on SC1-50, SC1-500 and SC2-500.

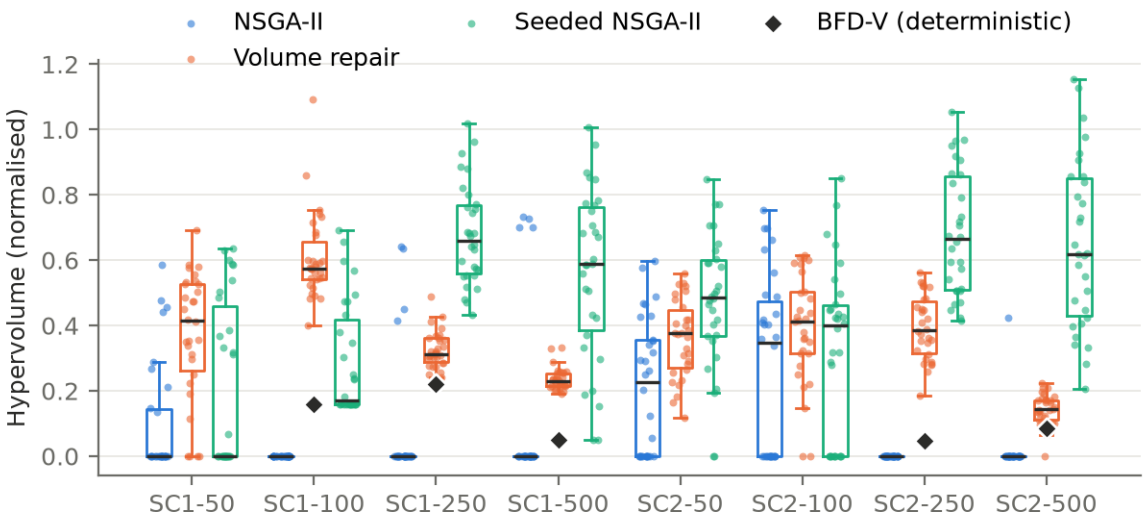


Figure 2: Hypervolume per run (30 seeds) for plain NSGA-II, the volume repair and seeded NSGA-II. Diamonds mark BFD-V where it is feasible.

6.2 Deterministic heuristics

Table 2. Items packed by the deterministic heuristics, one run each. An instance is feasible only if all N items are packed. HV is measured on the Table 1 reference front.

Instance	N	FF	BFD-V	BFD-P	BFD-V HV	Best MOEA HV
SC1-50	50	49	47	49	infeasible	0.371
SC1-100	100	97	**100**	97	0.160	0.608
SC1-250	250	248	**250**	248	0.222	0.679
SC1-500	500	499	**500**	494	0.051	0.560
SC2-50	50	49	49	48	infeasible	0.474
SC2-100	100	99	93	98	infeasible	0.391
SC2-250	250	246	**250**	245	0.046	0.682
SC2-500	500	498	**500**	497	0.086	0.651
SC3-50 / 100 / 250 / 500	—	45 / 91 / 202 / 459	38 / 82 / 187 / 382	48 / 88 / 228 / 452	infeasible	infeasible

BFD-V is feasible on five instances, and the best MOEA variant exceeds its hypervolume on every one of them. On SC1-50, SC2-50 and SC2-100, the MOEA variants find valid packings that none of the heuristics finds. No method packs every item on SC3. Because the generator controls volume rather than packability, whether complete packings exist there is unknown.

6.3 What the prior does to the trade-off

Table 1 compares runs that differ in feasibility. Table 3 restricts each comparison to seeds on which both methods are feasible, and compares hypervolume together with the best value each run reaches on each objective.

Table 3. Seeds on which both methods are feasible. Each cell gives the two methods' means, in the order named in the comparison. The best values are the per-run minimum CoM deviation (m) and maximum mean CASC, averaged over these seeds. p is from a seed-paired Wilcoxon test.

Instance	Comparison	Seeds	HV (p)	Best CoM deviation (p)	Best CASC (p)
SC1-50	repair vs NSGA-II	8	0.389 / 0.349 (0.64)	**0.019** / 0.032 (0.008)	0.257 / **0.304** (0.016)
SC2-50	repair vs NSGA-II	18	0.338 / 0.350 (0.80)	**0.014** / 0.027 (8 × 10^{-6})	0.230 / **0.286** (2.5 × 10^{-4})
SC2-100	repair vs NSGA-II	14	0.459 / 0.515 (0.30)	**0.056** / 0.071 (6 × 10^{-4})	0.212 / **0.248** (6 × 10^{-4})
SC2-50	seeded vs NSGA-II	16	**0.579** / 0.349 (4 × 10^{-4})	**0.014** / 0.027 (3 × 10^{-5})	0.234 / **0.285** (0.001)
SC2-100	seeded vs NSGA-II	12	0.482 / 0.548 (0.62)	0.071 / 0.070 (0.73)	0.244 / 0.249 (0.57)
SC1-50	repair vs seeded	12	0.410 / 0.438 (0.79)	**0.019** / 0.027 (0.005)	0.254 / **0.302** (0.003)
SC1-100	repair vs seeded	30	**0.608** / 0.291 (2 × 10^{-7})	**0.057** / 0.063 (3 × 10^{-4})	**0.189** / 0.176 (1 × 10^{-4})
SC2-50	repair vs seeded	28	0.349 / **0.507** (6 × 10^{-5})	0.014 / 0.015 (0.52)	0.228 / 0.236 (0.27)
SC2-100	repair vs seeded	18	0.426 / 0.478 (0.21)	**0.058** / 0.070 (0.002)	0.208 / **0.237** (0.002)
SC1-250	repair vs seeded	30	0.327 / **0.679** (2 × 10^{-9})	0.260 / **0.243** (2 × 10^{-9})	0.143 / **0.170** (3 × 10^{-8})
SC1-500	repair vs seeded	30	0.237 / **0.560** (1 × 10^{-6})	0.688 / **0.659** (6 × 10^{-4})	0.104 / **0.126** (2 × 10^{-7})
SC2-250	repair vs seeded	30	0.395 / **0.682** (5 × 10^{-7})	0.244 / **0.232** (2 × 10^{-5})	0.132 / **0.143** (9 × 10^{-4})
SC2-500	repair vs seeded	29	0.147 / **0.652** (4 × 10^{-9})	0.651 / **0.584** (4 × 10^{-9})	0.091 / **0.128** (4 × 10^{-9})
SC2-100	large-item vs volume repair	19	**0.528** / 0.410 (0.011)	0.069 / **0.058** (1 × 10^{-5})	**0.253** / 0.208 (1 × 10^{-4})

Three findings follow.

- **The repair shifts the front, it does not improve it.** On every instance where enough seeds are feasible for both, the repair's hypervolume is indistinguishable from plain NSGA-II's. Its fronts reach significantly lower CoM deviation and significantly lower accessibility. A plausible mechanism is that re-decoding from largest to smallest places the largest items first, at the access face. That keeps mass compact but buries small items behind large blockers. Section 6.7 shows the CoM gain does not hold on near-homogeneous cargo.
- **Seeding improves the front at scale.** On all four instances with 250 or 500 items, seeded NSGA-II beats the repair on both objectives in all eight comparisons, each significant. The repair forces the volume order onto every failed evaluation, whereas seeding offers it once and lets selection and recombination decide how much of it to keep. On the smaller instances the two priors trade off rather than one dominating. On SC1-50 and SC2-100, the repair reaches lower CoM deviation and seeding higher accessibility. On SC1-100 the repair is better on both objectives.
- **Partial ordering recovers accessibility, at a cost in feasibility.** On SC2-100, the large-item repair recovers the accessibility the full repair gives up (CASC 0.253 against 0.208) and has higher conditional hypervolume. It is feasible in fewer seeds (20 against 28), so its overall hypervolume in Table 1 is not higher.

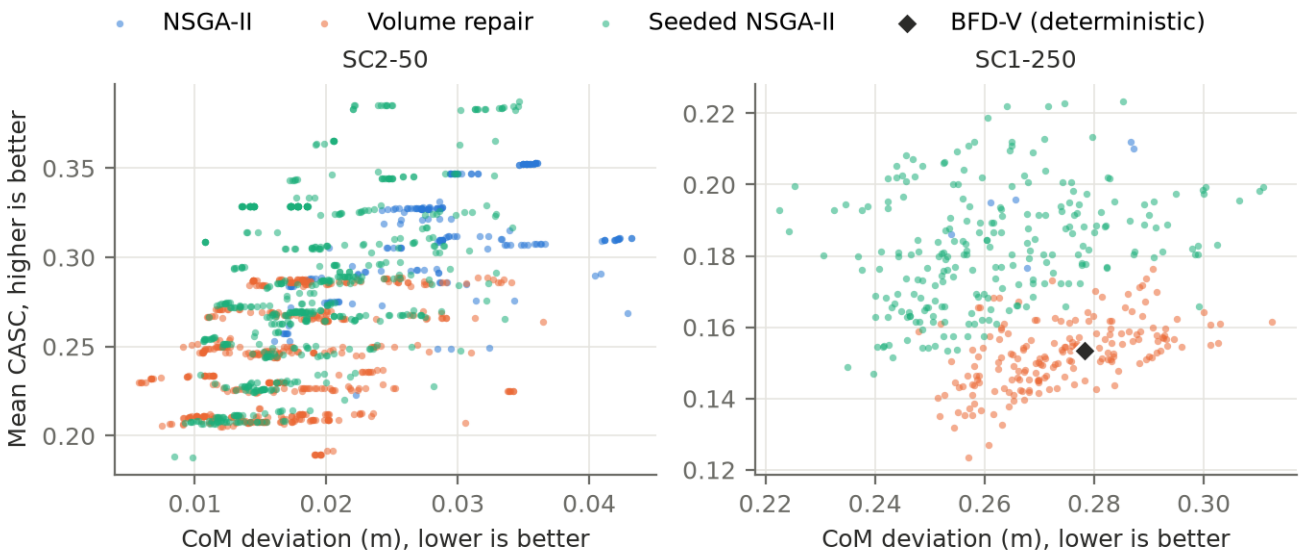


Figure 3: Pooled run fronts on SC2-50 (left) and SC1-250 (right). On SC2-50, NSGA-II and seeding reach higher CASC and the repair lower CoM deviation. On SC1-250, seeding dominates the repair, and NSGA-II is rarely feasible.

6.4 Sensitivity to the objective definition

The CASC constants and the CoM target were chosen by hand. For the volume repair against plain NSGA-II, we re-decoded and re-scored every Pareto point of every run under nine changed settings: each of L_arm, λ, C_grasp and κ halved and doubled, and the CoM target and dry mass moved to the container centre. BFD-V was re-run under each setting, and the statistics were recomputed on that comparison's own reference front. This analysis used the first study's seeds, 30 for $N \leq 100$ and 10 for $N \geq 250$. With the original constants it reproduces that study's statistics exactly.

Table 4. Robustness of the comparison between the repair and NSGA-II (eight instances).

Setting	Repair has higher mean HV	Holm-significant (α = 0.05)	Repair above BFD-V (of 5)
Original	8	7	5
L_arm × 0.5 / × 2	8 / 8	6 / 7	5 / 5
λ × 0.5 / × 2	8 / 8	7 / 7	5 / 5
C_grasp × 0.5 / × 2	8 / 8	6 / 7	5 / 5
κ × 0.5 / × 2	8 / 8	7 / 7	5 / 5
CoM target at container centre	7	6	5

No change to the accessibility constants alters the direction of the comparison. Moving the CoM target reverses one instance, SC2-50, where both methods are often feasible and the comparison depends on how CoM is defined. The analysis re-scores existing fronts rather than re-optimising, so it tests the robustness of the evaluation, not of the search. It does not cover the seeded variant.

6.5 Ablation of the volume repair

Six one-component ablations of the volume repair were run on SC1-100 and SC2-100 with seeds 1–10. Each ablated run's front was re-scored with the full objective and compared with the full repair, paired by seed, with Holm correction across the six ablations. HV here uses a reference front pooled over the ablation study and is not comparable with Table 1.

Table 5. Ablation results: HV mean ± SD, feasible seeds out of 10, and Holm-adjusted p against the full repair.

Variant	SC1-100 HV	Feas.	p	SC2-100 HV	Feas.	p
Full volume repair	0.758 ± 0.152	10	—	0.390 ± 0.193	9	—
Repair in random order	0	0	0.012	0.195 ± 0.259	4	0.64
No orientation search	0.028 ± 0.089	1	0.012	0.024 ± 0.051	2	0.023
Contact constraint	0.391 ± 0.136	10	0.012	0.064 ± 0.202	1	0.039
CASC without reach term	0.774 ± 0.162	10	0.85	0.397 ± 0.194	9	1.0
CASC without occlusion term	0.695 ± 0.119	10	0.11	0.395 ± 0.183	9	1.0
CASC without extraction term	0.661 ± 0.090	10	0.17	0.371 ± 0.164	9	1.0

- **The order is what matters.** A repair that re-decodes in random order is no better than no repair: 0 and 4 feasible seeds, against plain NSGA-II's 0 and 16 of 30. The gain comes from the decreasing-volume prior, not from a second decoding attempt.
- **Orientation search is essential.** With orientations fixed, feasibility collapses to 1 and 2 seeds out of 10.
- **No single CASC term drives the search.** Removing any one term from the search objective does not significantly change the full-objective front on these instances. This says nothing about whether each term is a meaningful model of retrieval effort.

6.6 Contact-constrained packing

The default decoder lets an item sit with a gap between it and the items nearer the access face. Under a contact constraint, each item away from the access face must rest at least half of its footprint on faces of items directly nearer the face. We re-ran every method with this constraint on the four instances with N ≤ 100 (10 seeds each, 204 runs).

Table 6. Contact-constrained decoder: HV mean ± SD and feasible seeds out of 10. BFD-V uses the same contact rule. Random search was infeasible in every run.

Instance	NSGA-II	Volume repair	Seeded NSGA-II	Large-item repair	BFD-V
SC1-50	0 · 0	**0.161 ± 0.222** · 5	0.116 ± 0.264 · 2	0.020 ± 0.063 · 1	infeasible
SC1-100	0 · 0	**0.478 ± 0.144** · 10	0.267 ± 0.175 · 10	0.414 ± 0.315 · 9	0.185
SC2-50	0.181 ± 0.214 · 5	0.206 ± 0.206 · 6	**0.335 ± 0.095** · 10	0.141 ± 0.156 · 5	0.158
SC2-100	0.171 ± 0.384 · 3	0.074 ± 0.234 · 1	**0.240 ± 0.261** · 5	0.158 ± 0.264 · 3	infeasible

The contact constraint sharply reduces feasibility for every method. Only on SC1-100 are both prior-injecting methods significantly better than plain NSGA-II (Holm p = 0.008 for each). There the repair leads seeding (uncorrected p = 0.049, Holm 0.20). On SC2 seeding has the best mean, but with ten seeds no difference survives correction. The qualitative picture holds under the stricter physics: a constructive prior is needed for feasibility, and repair and seeding each win on different instances. The magnitudes do not hold, and the unconstrained results should not be read as predictions for a physically constrained packer.

6.7 External benchmark: Bischoff–Ratcliff instances

To test the findings outside our generator, we adapted the container-loading instances of Bischoff and Ratcliff (1995), distributed through OR-Library as `thpack1`–`thpack7`. We used problems 1 and 2 of three classes that span cargo heterogeneity: BR1 with 3 box types, BR4 with 10, and BR7 with 20. The container is the benchmark's 587 × 233 × 220 cm box with its door as the access face, so z runs along the 5.87 m length and y is vertical.

The benchmark targets volume maximisation, and each problem holds about one container's volume of boxes. Our model requires every item to be placed, so we adapted each problem in four steps, fixed before any run:

1. **Subset.** The boxes were expanded, shuffled with the problem's own BR seed, and taken up to 80% of container volume, matching SC1 and SC2. This gives 88 to 111 boxes per instance.
2. **Orientation.** Each box keeps its BR orientation flags, so a dimension may be vertical only if the benchmark permits it. Between 34 and 73 boxes per instance are restricted.
3. **Mass and category.** BR boxes have neither. Each box type was given a category, drawn with SC1's probabilities, and a density uniform on [100, 300] kg/m³. Each box's mass is that density times its volume, with lognormal noise ($\sigma = 0.2$). The masses are therefore synthetic.
4. **CoM target.** The container centre.

Every method was run with seeds 1–10 (306 runs), and Holm correction was applied across the six instances.

Table 7. Bischoff–Ratcliff instances: HV as the mean with its bootstrap 95% interval, and feasible seeds out of 10. The best mean per row is in bold. BFD-V shows its HV, or the number of boxes it packs if infeasible.

Instance	Types · N	NSGA-II	Volume repair	Seeded NSGA-II	Large-item repair	BFD-V
BR1-1	3 · 90	**0.447** [0.242, 0.637] · 7	0.186 [0.156, 0.215] · 10	0.326 [0.171, 0.482] · 7	0.186 [0.157, 0.216] · 10	88/90 packed
BR1-2	3 · 111	0.774 [0.659, 0.877] · 10	0.724 [0.682, 0.776] · 10	**0.791** [0.730, 0.849] · 10	0.724 [0.683, 0.777] · 10	0.138
BR4-1	10 · 88	0.358 [0.121, 0.624] · 5	**0.416** [0.348, 0.497] · 10	0.370 [0.279, 0.464] · 10	0.401 [0.367, 0.435] · 10	86/88 packed
BR4-2	10 · 103	0.068 [0.000, 0.163] · 2	0.348 [0.305, 0.393] · 10	**0.550** [0.461, 0.637] · 10	0.206 [0.164, 0.246] · 10	0.010
BR7-1	20 · 94	0 · 0	0.409 [0.203, 0.624] · 7	0.179 [0.000, 0.438] · 2	**0.449** [0.297, 0.595] · 10	85/94 packed
BR7-2	20 · 111	0 · 0	**0.289** [0.109, 0.529] · 8	0.081 [0.000, 0.178] · 3	0.117 [0.050, 0.193] · 6	100/111 packed

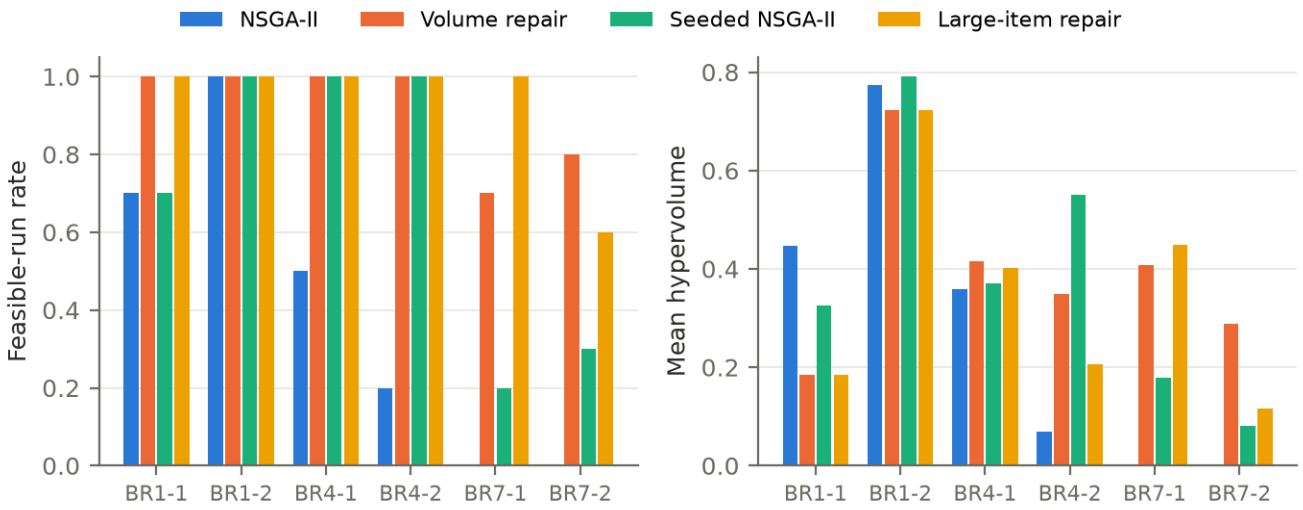


Figure 4: Feasible-run rate (left) and mean hypervolume (right) on the Bischoff–Ratcliff instances. Feasibility for plain NSGA-II falls as the number of box types grows from 3 to 20.

Heterogeneity decides whether a prior is needed. With 3 box types, plain NSGA-II is feasible in 7 and 10 of 10 seeds and has the best or near-best hypervolume. The volume repair lowers front quality on BR1-1, where its mean HV is 0.186 against 0.447 ($\delta = -0.40$, Holm $p = 0.082$). Restricted to the seven seeds where both are feasible, the repair's hypervolume is significantly lower (0.207 against 0.638, $p = 0.016$), and so is its accessibility (CASC 0.110 against 0.156, $p = 0.016$). Its CoM deviation is higher as well (0.135 against 0.079 m, $p = 0.047$). On near-uniform cargo the volume order carries no useful information and only constrains the search.

With 10 box types, both priors restore feasibility. On BR4-2 seeding is best, significantly better than both the repair (Holm $p = 0.023$) and NSGA-II ($p = 0.012$).

With 20 box types, plain NSGA-II never finds a valid packing, and BFD-V and the seed are infeasible. Here the repairs are necessary. The volume repair beats NSGA-II on BR7-2 (Holm $p = 0.047$), and the large-item repair beats it on BR7-1 ($p = 0.012$). Seeding, whose seed is infeasible, is feasible in only 2 and 3 of 10 seeds.

These results agree with the synthetic study where the two overlap and qualify it where they do not: - A constructive prior is needed for feasibility once cargo is heterogeneous, and can be harmful when it is not. - Seeding is the stronger prior when its seed is feasible, and the repair is the more robust prior when no constructive solution is feasible. - The repair's CoM advantage seen in Table 3 is not universal.

6.8 Computational cost

The volume repair and the large-item repair use almost two decoder calls per evaluation. Seeded NSGA-II uses one, like plain NSGA-II. In the first study, the volume repair took 1.6–1.75 times NSGA-II's wall-clock time on every instance. The later runs were executed with more parallel workers, so their wall-clock times are not comparable and are not reported, but their decoder-call counts are exact. A time-matched comparison would give the single-decode methods about 1.7 times as many evaluations. That favours seeding further, and would not rescue plain NSGA-II, which is infeasible in nearly every seed on five synthetic instances and both 20-type BR instances.

On an NVIDIA RTX 4050 Laptop GPU, the corrected batched scorer evaluates 50, 100 and 250 packed SC1-100 layouts in 0.016, 0.041 and 0.108 s, against 0.101, 0.227 and 0.449 s on the CPU. These are

medians of five repeats after warm-up, giving speedups of 6.4×, 5.5× and 4.1×. Packing remains on the CPU, and the whole-run speedup was not measured.

7. Discussion

Report feasibility before front quality. Hypervolume against a feasible reference front assigns 0 to a run without a valid packing. A method that is often infeasible therefore loses even when its feasible fronts are as good. Table 1 alone would suggest that the volume repair finds better trade-offs than plain NSGA-II. Table 3 shows that it does not: it finds valid packings, and among valid packings it occupies a different region of the front. Evaluations of decoder-based packing MOEAs should report feasible-run rates and a feasibility-conditioned comparison alongside overall hypervolume.

How the prior is injected matters. The volume repair and seeding carry the same constructive knowledge but behave differently. The repair applies the order to every failed evaluation, overriding the search whenever a packing would fail. Seeding supplies the order once, and the search can recombine it with other orderings. When a feasible seed exists, which in our data means every synthetic instance with 250 or more items and BR4-2, seeding wins on both objectives. When none exists, as on the 20-type BR instances, the repair keeps working and seeding fails. Seeding is also simpler and costs no extra decoding. A practical rule from our data is to seed when a constructive heuristic finds a feasible packing, and to repair when it does not. That rule is an inference from 14 solvable instances and should be tested more widely.

Whether a prior is needed depends on the cargo. On near-homogeneous cargo (BR1), the decoder already finds valid packings and the volume order constrains the search without helping it. On heterogeneous cargo (SC1, SC2, BR7), the decoder fails without help. Practitioners should check plain NSGA-II's feasibility before adding a prior.

Repairs carry a bias. A decreasing-volume repair imports BFD-V's logic. That is why it restores feasibility, and on most instances also why it trades accessibility for compactness. The large-item repair shows the bias can be reduced, at some cost in feasibility. Priors whose order reflects accessibility, for example placing frequently retrieved items nearest the access face, are a natural next step.

Strong constructive baselines are necessary. BFD-V alone is feasible on seven of the eighteen instances, including five synthetic ones where plain NSGA-II rarely or never is. A study comparing an MOEA only with random search would overstate the value of the search. In our data the MOEA variants add front quality over BFD-V on every instance where BFD-V is feasible, and find valid packings where it is not.

Check numerical equivalence on the layouts the search visits. The scorer defect in Section 5 was invisible to an equivalence check on fully packed layouts. It appeared only on the partially packed layouts that dominate search.

8. Threats to Validity and Limitations

Internal validity.

- The ablation, contact and Bischoff–Ratcliff studies use 10 seeds on four or six instances.
- Wall-clock times were taken under different parallel loads, so we compare decoder calls rather than time for the later methods.
- Hyperparameters were not tuned for any method.
- The deterministic heuristics are single runs and are compared descriptively.
- The sensitivity analysis re-scores existing fronts of the comparison between the repair and NSGA-II rather than re-optimising, and does not cover the seeded variant.
- Hypervolume values depend on the pooled method set, so they are compared only within a study.

Construct validity. CASC is an unvalidated surrogate for retrieval effort, and the CoM model is a rigid-body approximation. The main comparison is robust to the tested perturbations of both, but the trade-off has not been validated against crew or mission data. On the Bischoff–Ratcliff instances the masses and categories are synthetic, and the reach term saturates over most of a 5.87 m deep container, which flattens the accessibility objective there.

External validity.

- The Bischoff–Ratcliff instances are subsets adapted to a complete-packing model, so our results are not comparable with published volume-utilisation results on that benchmark.
- Only six of its problems, from three of its classes, were used. The strongly heterogeneous classes BR8–BR15 were not tested.
- Launch-load restraint, static equilibrium (Ramos et al., 2016) and soft goods are not modelled, and the contact study shows that stricter physics changes the results.
- Only one MOEA and one decoder family were studied.

Reproducibility. All code, instance files (including the Bischoff–Ratcliff source data and the adaptation script), derived results and figures are in the repository. Objective values were reproduced exactly in a second environment. The raw evaluation archives (about 2 GB) are regenerated by the commands below.

9. Conclusion

In a bi-objective CoM–accessibility packing model, plain NSGA-II with an EP decoder rarely returns a valid packing on heterogeneous cargo. Injecting a decreasing-volume constructive prior fixes this, either through a Baldwinian repair or through a single seeded individual. Over 30 seeds, the repair beats plain NSGA-II in hypervolume on all eight solvable synthetic instances and seeding on six, and both beat the deterministic BFD-V construction wherever it is feasible.

The two ways of injecting the prior are not equivalent. Seeding is significantly better on five instances, including every instance with 250 or more items, where it improves both objectives over the repair at half the decoding cost. The repair is better on the two smallest consumable-heavy instances, and it is the only prior that works when no constructive solution is feasible.

Among feasible runs, the repair's gain over NSGA-II is a feasibility gain rather than a better front. It shifts solutions toward CoM compactness at the expense of accessibility. On an external benchmark, the prior is essential for heterogeneous cargo and harmful for near-uniform cargo. Ablation attributes the gain to the volume order, and a contact constraint lowers feasibility for every method without removing the need for a prior.

Evaluations of packing optimisers should report feasibility separately from front quality and include strong constructive baselines. Whether a prior is needed, and how it is injected, should be decided by testing. The next steps are accessibility-aware priors, contact- and equilibrium-aware decoders, the remaining benchmark classes, and validation of the surrogates against operational data.

Reproducibility Statement

The code, instances and analysis scripts are in the project repository. The derived data behind every table and figure are in `docs/paper/data/`: per-run metrics, extracted Pareto fronts, and the sensitivity, ablation, contact and Bischoff–Ratcliff results. The test suite (`pytest tests`, 50 tests) covers: - the scorer correction; - every method variant; - the seeded chromosome; - the contact and ablation decoders; - the Bischoff–Ratcliff orientation flags.

Run from the repository root:

```
# Main study: NSGA-II, volume repair, random search
python experiments/matched_budget_runner.py \
```

```
    --datasets SC1-50 SC2-50 SC3-50 SC1-100 SC2-100 SC3-100 \
    --seeds 1 ... 30 --out experiments/full_matched
python experiments/matched_budget_runner.py \
    --datasets SC1-250 SC2-250 SC3-250 SC1-500 SC2-500 SC3-500 \
    --seeds 1 ... 10 --out experiments/full_matched
python experiments/extended_runner.py --stage more_seeds --workers 14
python experiments/heuristic_baselines.py --out experiments/full_matched

# Seeded and large-item variants; contact study; Bischoff-Ratcliff study
python experiments/extended_runner.py --stage new_methods --workers 14
python experiments/extended_runner.py --stage contact --workers 14
python experiments/br_instances.py
python experiments/extended_runner.py --stage br --workers 14

# Analysis (repeat for experiments/contact and experiments/br)
python experiments/matched_budget_analysis.py --dir experiments/full_matched
python experiments/conditional_analysis.py \
    --metrics experiments/full_matched/per_run_metrics.csv

# Sensitivity, ablation, figures (D = docs/paper/data)
python experiments/sensitivity_rescore.py \
    --fronts $D/fronts_full_matched.npz --out experiments/sensitivity
python experiments/ablation_study.py --datasets SC1-100 SC2-100 \
    --seeds 1 2 3 4 5 6 7 8 9 10 --workers 12
python experiments/ablation_analysis.py --ablation experiments/ablation \
    --fronts $D/fronts_full_matched.npz
python experiments/paper_figures.py --metrics $D/v8/per_run_metrics.csv \
    --fronts $D/v8/fronts_v8.npz --baselines experiments/full_matched \
    --br-metrics $D/br/per_run_metrics.csv

# Scoring-stage GPU timing (CUDA device required)
python experiments/benchmark_gpu_v2.py
```

Appendix A. Relation to an earlier analysis of this system

An earlier version of this work reported that NSGA-II significantly outperformed single-construction heuristics in hypervolume, and claimed a whole-run GPU speedup. Those results are withdrawn, for four reasons:

1. They were computed with the defective batched scorer (Section 5), which biased the accessibility objective for every infeasible layout.
2. Most reported solutions were infeasible.
3. The significance test compared 30 stochastic hypervolumes with a single fixed baseline value. That test asks whether every run exceeds the value, not whether the methods differ.
4. The GPU benchmark timed packing and scoring together, including start-up, and its equivalence check compared empty outputs.

This paper replaces those analyses with the protocol of Section 4 and the scoring-stage measurement of Section 6.8.

References

- Bischoff, E. E., and Ratcliff, M. S. W. (1995). Issues in the development of approaches to container loading. *Omega*, 23(4), 377–390.

- Blank, J., and Deb, K. (2020). pymoo: Multi-objective optimization in Python. *IEEE Access*, 8. DOI 10.1109/ACCESS.2020.2990567.
- Bortfeldt, A., and Wäscher, G. (2013). Constraints in container loading – a state-of-the-art review. *European Journal of Operational Research*, 229(1), 1–20. DOI 10.1016/j.ejor.2012.12.006.
- Ceschia, S., and Schaerf, A. (2013). Local search for a multi-drop multi-container loading problem. *Journal of Heuristics*, 19(2), 275–294. DOI 10.1007/s10732-011-9162-6.
- Cliff, N. (1993). Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3), 494–509. DOI 10.1037/0033-2909.114.3.494.
- Crainic, T. G., Perboli, G., and Tadei, R. (2008). Extreme point-based heuristics for three-dimensional bin packing. *INFORMS Journal on Computing*, 20(3), 368–384. DOI 10.1287/ijoc.1070.0250.
- Davies, A. P., and Bischoff, E. E. (1999). Weight distribution considerations in container loading. *European Journal of Operational Research*, 114(3), 509–527.
- Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338. DOI 10.1016/S0045-7825(99)00389-8.
- Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197. DOI 10.1109/4235.996017.
- Derrac, J., García, S., Molina, D., and Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1), 3–18. DOI 10.1016/j.swevo.2011.02.002.
- Fasano, G. (2014). *Solving Non-standard Packing Problems by Global Optimization and Heuristics*. SpringerBriefs in Optimization. Springer. DOI 10.1007/978-3-319-05005-8.
- Fasano, G., Lavopa, C., Negri, D., and Vola, M. C. (2015). CAST: A successful project in support of the International Space Station logistics. In *Optimized Packings with Applications*, Springer Optimization and Its Applications, 87–117. DOI 10.1007/978-3-319-18899-7_5.
- Friedrich, T., and Wagner, M. (2015). Seeding the initial population of multi-objective evolutionary algorithms: A computational study. *Applied Soft Computing*, 33, 223–230. DOI 10.1016/j.asoc.2015.04.043.
- Gonçalves, J. F., and Resende, M. G. C. (2013). A biased random key genetic algorithm for 2D and 3D bin packing problems. *International Journal of Production Economics*, 145(2), 500–510. DOI 10.1016/j.ijpe.2013.04.019.
- Ho, K. (2023). Space logistics modeling and optimization: Review of the state of the art. arXiv:2306.01107. DOI 10.2514/1.A35982.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Martello, S., Pisinger, D., and Vigo, D. (2000). The three-dimensional bin packing problem. *Operations Research*, 48(2), 256–267. DOI 10.1287/opre.48.2.256.12386.
- Özcan, S. O., Dokeroğlu, T., Coşar, A., and Yazıcı, A. (2016). A novel grouping genetic algorithm for the one-dimensional bin packing problem on GPU. In *Computer and Information Sciences (ISCIS 2016)*. DOI 10.1007/978-3-319-47217-1_6.
- Parreño, F., Alvarez-Valdes, R., Tamarit, J. M., and Oliveira, J. F. (2008). A maximal-space algorithm for the container loading problem. *INFORMS Journal on Computing*, 20(3), 412–422. DOI 10.1287/ijoc.1070.0254.
- Ramos, A. G., Oliveira, J. F., Gonçalves, J. F., and Lopes, M. P. (2016). A container loading algorithm with static mechanical equilibrium stability constraints. *Transportation Research Part B: Methodological*, 91, 565–581. DOI 10.1016/j.trb.2016.06.003.
- Salcedo-Sanz, S. (2009). A survey of repair methods used as constraint handling techniques in evolutionary algorithms. *Computer Science Review*. DOI 10.1016/j.cosrev.2009.07.001.
- Whitley, D., Gordon, V. S., and Mathias, K. (1994). Lamarckian evolution, the Baldwin effect and function optimization. In *Parallel Problem Solving from Nature – PPSN III*, LNCS 866. Springer.

DOI 10.1007/3-540-58484-6_245.
- Zhao, X., Bennell, J. A., Bektaş, T., and Dowsland, K. (2016). A comparative review of 3D container loading algorithms. *International Transactions in Operational Research*, 23(1–2), 287–320. DOI 10.1111/itor.12094.
- Zitzler, E., and Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4), 257–271. DOI 10.1109/4235.797969.
- Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C. M., and da Fonseca, V. G. (2003). Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2), 117–132. DOI 10.1109/TEVC.2003.810758.